PROJECT ACTIVITY REPORT

Activity 4.1: Multimodal Agent Implementation

Project: Multimodal Attendance & Engagement Agent

1. Milestone Overview & Objective

This milestone implements the **Multimodal Agent**, the core intelligence layer of the system. Instead of relying on isolated single-signal measurements that often misread classroom dynamics, the agent combines the continuous outputs of the Vision Module and the Audio Module into one context-aware picture of classroom behavior.

Primary engineering goal. Engagement estimation and session analysis must be grounded in contextual, interpretable and explainable logic. The agent applies rule-based reasoning first and uses structured LLM synthesis second, so the output is trustworthy enough for real educational settings.

2. Multimodal Fusion & Rule-Based Reasoning Architecture

Single-modality pipelines produce many false positives. A student looking down to take notes can be classed as disengaged, and lively group discussion can be treated as disruptive noise. The agent avoids this by synchronizing time windows across both perception streams and evaluating deterministic rules *before* any generative summary is written.

1. PERCEPTION INPUTS	2. WINDOW FUSION	3. RULE REASONING	4. LLM INSIGHT
Vision + Audio modules	Align time windows	Deterministic matrix	InsightGenerator
`gaze, pose, VAD`	`vision / audio dicts`	`state + reason`	`insight + advice`

Figure 1: Multimodal agent flow. The LLM only verbalizes what the rule layer has already decided.

2.1 Input Modality Signals

Module	Signals provided
Vision Module	Real-time attendance headcount, facial presence verification, gaze direction vectors, head-pose estimation (pitch, yaw, roll), and posture / attentiveness indicators.
Audio Module	Voice Activity Detection (VAD) ratios, speaker activity states (instructor lecture vs. student participation), ambient acoustic energy levels, and silence / discussion intervals.

2.2 Contextual Rule-Based Reasoning Matrix

The agent applies deterministic cross-modal rules that map combined visual and acoustic states to explainable behavioral categories.

Vision signal state	Audio signal state	Inferred classroom state	Explainable reasoning logic
High forward gaze and stable head pose	Sustained instructor speech (active VAD)	**Focused Lecture** (High engagement)	Students keep visual focus on the presentation area while the instructor delivers content.
Varied gaze angles and frequent head turns	Multi-speaker turn-taking and elevated energy	**Interactive Discussion** (High engagement)	Gaze shifts match peer-to-peer verbal exchange rather than distraction.
Downward pitch (looking at desk)	Low vocal activity or periodic instructor speech	**Note-Taking / Individual Task** (Moderate-High)	Downward head pose during quiet or dictation intervals indicates writing, not fatigue.
Averted gaze, slouching or missing faces	Prolonged unprompted silence or off-topic noise	**Passive / Disengaged** (Low engagement)	A simultaneous drop in visual attention and structured acoustic participation signals attention drift.

3. Core Agent & LLM Insight Generation Implementation

Structured multimodal telemetry is turned into short, actionable feedback for instructors by the `InsightGenerator` service in `backend/core/llm_insights.py`. It places the rule-based session metrics into a structured prompt, queries the language model, and falls back to safe defaults when the response is unusable. The rule layer itself lives in a separate module so it can be tested without any model.

3.1 Rule Engine

The matrix from Section 2.2 is implemented as ordered rules. Order matters: Disengaged is checked first, and it requires *both* weak visual attention and weak acoustic structure. Looking down alone does not count as weak attention, which is what protects note-taking from being misread.

File: backend/core/multimodal_rules.py (reference implementation)

```
# File: backend/core/multimodal_rules.py
from dataclasses import dataclass

# Starting-point thresholds: calibrate them on your own classroom recordings.
T = {
    "forward_high": 0.65,     # share of students facing the presentation area
    "forward_low": 0.35,
    "turn_stable": 3.0,       # head turns per minute (class average)
    "turn_varied": 6.0,
    "down_high": 0.50,        # share of students looking down at the desk
    "missing_high": 0.30,     # share of expected faces not visible
    "slouch_high": 0.40,
    "instructor_active": 0.50,  # share of the window with instructor speech
    "turns_discussion": 4.0,  # speaker changes per minute
    "energy_elevated": 0.50,  # normalized ambient energy (0-1)
    "vad_low": 0.40,
    "silence_long": 0.60,
}


@dataclass
class RuleResult:
    state: str
    engagement: str
    reason: str


def classify(vision: dict, audio: dict) -> RuleResult:
    """Deterministic cross-modal rules; returns a label plus the reason."""
    fwd = vision.get("forward_gaze_ratio", 0.0)
    turns = vision.get("head_turn_rate", 0.0)
    down = vision.get("downward_pitch_ratio", 0.0)
    missing = vision.get("missing_face_ratio", 0.0)
    slouch = vision.get("slouch_ratio", 0.0)

    vad = audio.get("vad_ratio", 0.0)
    instr = audio.get("instructor_speech_ratio", 0.0)
    changes = audio.get("speaker_changes_per_min", 0.0)
    energy = audio.get("energy", 0.0)
    silence = audio.get("silence_ratio", 0.0)
    off_topic = audio.get("off_topic_noise", False)

    # 1. Passive / Disengaged: weak visual attention AND weak acoustic structure
    vision_low = (missing >= T["missing_high"] or slouch >= T["slouch_high"]
```

```
                or (fwd < T["forward_low"] and down < T["down_high"]))
    audio_passive = silence >= T["silence_long"] or off_topic
    if vision_low and audio_passive:
        return RuleResult("Passive / Disengaged", "Low",
            "Visual attention dropped (averted gaze, slouching or missing faces) "
            "while the room showed prolonged silence or off-topic noise.")

    # 2. Focused Lecture: forward gaze, stable heads, instructor speaking
    if (fwd >= T["forward_high"] and turns <= T["turn_stable"]
            and instr >= T["instructor_active"]):
        return RuleResult("Focused Lecture", "High",
            "Students keep visual focus on the presentation area while the "
            "instructor speaks continuously.")

    # 3. Interactive Discussion: varied gaze plus multi-speaker turn-taking
    if (turns >= T["turn_varied"] and changes >= T["turns_discussion"]
            and energy >= T["energy_elevated"]):
        return RuleResult("Interactive Discussion", "High",
            "Gaze shifts and head turns coincide with multi-speaker exchange "
            "and elevated energy, which points to peer discussion.")

    # 4. Note-Taking / Individual Task: heads down while the room is quiet
    if down >= T["down_high"] and vad <= T["vad_low"]:
        return RuleResult("Note-Taking / Individual Task", "Moderate-High",
            "Downward head pose during quiet or dictation intervals suggests "
            "writing rather than fatigue.")

    # 5. No rule matched: report it honestly instead of guessing
    return RuleResult("Mixed / Undetermined", "Moderate",
        "Signals do not match a defined pattern.")
```

3.2 InsightGenerator

The original reference listing ends mid-function (at `return {`), and it does not show the constructor or the client. The version below completes it. The prompt and generation parameters (`temperature=0.5`, `max_tokens=150`) are kept from the original; changes are listed after the code.

File: backend/core/llm_insights.py (completed reference implementation)

```
# File: backend/core/llm_insights.py
import os

DEFAULT_INSIGHT = "Class status is stable."
DEFAULT_RECOMMENDATION = "Maintain current pace."

SYSTEM_PROMPT = (
    "You are an expert educational AI assistant. Provide brief, professional, "
    "and actionable insights for classroom engagement. Use only the telemetry "
    "given; do not invent numbers or events. Reply in exactly two lines:\n"
    "Insight: <one sentence>\n"
    "Recommendation: <one sentence>"
)


class InsightGenerator:
    def __init__(self, client, model=None):
        # client: any OpenAI-compatible chat client (chat.completions.create)
        self.client = client
        self.model = model or os.getenv("LLM_MODEL", "")

    def _build_prompt(self, d: dict) -> str:
        return (
            "Session Telemetry Summary:\n"
            f"- Attendance Count: {d.get('attendance_count', 0)}\n"
            f"- Visual Attention Score: {d.get('visual_attention', 0.0)}\n"
            f"- Audio Participation State: {d.get('audio_state', 'normal')}\n"
            f"- Rule-Based Classification: {d.get('rule_state', 'stable')}\n"
            f"- Rule Reasoning: {d.get('rule_reason', 'n/a')}\n"
            "Provide a concise classroom status insight and one actionable "
            "recommendation."
        )

    @staticmethod
    def _parse(text: str):
        insight, recommendation = DEFAULT_INSIGHT, DEFAULT_RECOMMENDATION
        found = set()
        for line in (text or "").splitlines():
            low = line.strip().lower()
            if low.startswith("insight:") and line.split(":", 1)[1].strip():
                insight = line.split(":", 1)[1].strip()
                found.add("i")
            elif low.startswith("recommendation:") and line.split(":", 1)[1].strip():
                recommendation = line.split(":", 1)[1].strip()
```

```
                found.add("r")
        return insight, recommendation, len(found) == 2

    def generate_classroom_insight(self, session_data: dict) -> dict:
        """Verbalizes rule-based findings; never changes the rule label."""
        source = "llm"
        try:
            completion = self.client.chat.completions.create(
                messages=[{"role": "system", "content": SYSTEM_PROMPT},
                          {"role": "user", "content": self._build_prompt(session_data)}],
                model=self.model,
                temperature=0.5,
                max_tokens=150,
                timeout=10,
            )
            insight, recommendation, ok = self._parse(completion.choices[0].message.content)
            source = "llm" if ok else "llm_unparsed"   # defaults used for missing parts
        except Exception as e:      # timeouts, rate limits, bad model ID, etc.
            print(f"LLM insight error: {e}")
            insight, recommendation, source = DEFAULT_INSIGHT, DEFAULT_RECOMMENDATION, "fallback"
        return {"insight": insight,
                "recommendation": recommendation,
                "rule_state": session_data.get("rule_state", "stable"),
                "source": source}
```

Changes from the source listing

- **Client injection:** the constructor takes any OpenAI-compatible client, so the LLM provider can be swapped or mocked in tests.
- **Configurable model:** the model ID is read from the `LLM_MODEL` environment variable (see Section 5).
- **Real parsing:** the system prompt asks for `Insight:` and `Recommendation:` lines, which are parsed. In the original listing the response was read but the default strings were returned.
- **Rule reasoning in the prompt:** the rule engine's explanation is passed to the LLM so it can verbalize *why* the state was chosen.
- **Status flag and timeout:** the returned `source` is `llm`, `llm_unparsed` or `fallback`, and the call has a 10 s timeout.

Behaviour check

The rule engine and generator were run with synthetic inputs and a mock client (no real model was called). Five test windows produced Focused Lecture, Interactive Discussion, Note-Taking, Passive / Disengaged and Mixed / Undetermined as expected. A well-formed reply was parsed, a malformed reply returned the defaults with status `llm_unparsed`, and a raised timeout returned the defaults with status `fallback`. It has not been tested with real classroom data or a live LLM.

3.3 Wiring the Agent

```
# File: backend/core/multimodal_agent.py
from backend.core.multimodal_rules import classify


class MultimodalAgent:
    def __init__(self, insight_generator):
        self.insights = insight_generator

    def analyze_window(self, attendance_count, vision, audio):
        rule = classify(vision, audio)                    # deterministic first
        session_data = {
            "attendance_count": attendance_count,
            "visual_attention": vision.get("forward_gaze_ratio", 0.0),
            "audio_state": "active" if audio.get("vad_ratio", 0) > 0.4 else "quiet",
            "rule_state": rule.state,
            "rule_reason": rule.reason,
        }
        out = self.insights.generate_classroom_insight(session_data)
        out.update({"engagement": rule.engagement, "reason": rule.reason})
        return out
```

Example output shape (illustrative values):

```
{"insight": "Students are following the lecture closely.",
 "recommendation": "Add a quick check-for-understanding question.",
 "rule_state": "Focused Lecture", "engagement": "High",
 "reason": "Students keep visual focus on the presentation area ...",
 "source": "llm"}
```

4. Architectural Design Decisions

• **Explainability first:** the LLM does not guess engagement scores from scratch. Rule-based multimodal reasoning computes verified state labels first, and the LLM puts those findings into clear guidance for the educator.

• **Controlled generation parameters:** `temperature=0.5` and `max_tokens=150` balance low latency with short, focused output for live dashboards. Grounding comes mainly from the rule label and the constrained prompt; low temperature alone does not prevent errors.

• **Fail-safe defaults:** the baseline strings `"Class status is stable."` and `"Maintain current pace."` keep telemetry reporting uninterrupted during API timeouts.

• **Label stays with the rules:** the returned `rule_state` comes from the rule engine, never from the LLM text, so the dashboard label cannot be changed by a model response.

5. Model Configuration Note

The source document specifies `mixtral-8x7b-32768`. That ID belongs to Groq's hosted catalogue, and public Groq deprecation notices and changelogs state it was retired on 20 March 2025, so calls using it are expected to fail with a model-not-found style error. Because of this the reference code does not hard-code a model.

Choose a currently supported model from your provider's model list and set it through the environment. Model catalogues change often, so verify the ID before each release. If the call fails, the fail-safe defaults still keep the dashboard running, but every insight will then be the generic default text, so monitor the `source`

field.

```
# .env
LLM_MODEL=<current-model-id-from-your-provider>
```

6. Rule Threshold Reference

The thresholds in `multimodal_rules.py` are starting values chosen for illustration; they are not validated on classroom data and must be calibrated.

Threshold	Start value	Used by
forward_high / forward_low	0.65 / 0.35	Share of students facing the presentation area (Lecture, Disengaged).
turn_stable / turn_varied	3 / 6 turns per min	Head-turn rate separating stable from varied gaze.
down_high	0.50	Share looking down; supports Note-Taking and protects it from Disengaged.
missing_high / slouch_high	0.30 / 0.40	Missing faces and slouching (Disengaged).
instructor_active	0.50	Instructor speech share required for Focused Lecture.
turns_discussion / energy_elevated	4 per min / 0.50	Speaker changes and energy required for Discussion.
vad_low / silence_long	0.40 / 0.60	Quiet-room tests for Note-Taking and Disengaged.

7. Limitations & Responsible Use

Limitation	Impact	Mitigation
Thresholds are uncalibrated	Wrong labels in real rooms.	Calibrate on labelled recordings; review confusion between states.
Group-level signals only	Cannot say whether one individual is engaged.	Report class-level states; avoid individual scoring or discipline use.
Posture and gaze are imperfect engagement proxies	Neurodiverse or tired students may be misread.	Keep a human in the loop; present results as advice, not judgement.
LLM text can still be wrong or vague	Misleading advice on the dashboard.	Show the rule label and reason next to the text; keep prompts constrained.
Model IDs and APIs change	Calls start failing after retirements.	Configurable model, monitor the source field, add alerting.
Audio and video of students are personal data	Privacy and consent obligations.	Obtain consent, store only derived metrics where possible, define retention.

8. Validation Checklist

Tick each item after running it on your machine:

Check	How to verify	Status
Focused Lecture detected	Forward gaze high, instructor speaking: state is Focused Lecture	[ ]

Check	How to verify	Status
Discussion not marked disruptive	Many speakers and head turns: state is Interactive Discussion	[ ]
Note-taking not marked disengaged	Heads down, quiet room: state is Note-Taking	[ ]
Disengagement detected	Missing faces plus long silence: state is Passive / Disengaged	[ ]
Prompt contains rule label	Inspect the prompt sent to the model	[ ]
Fallback works	Use a wrong model ID or no network: defaults returned, source = fallback	[ ]
Live model call works	Valid LLM_MODEL set: source = llm and text parsed	[ ]